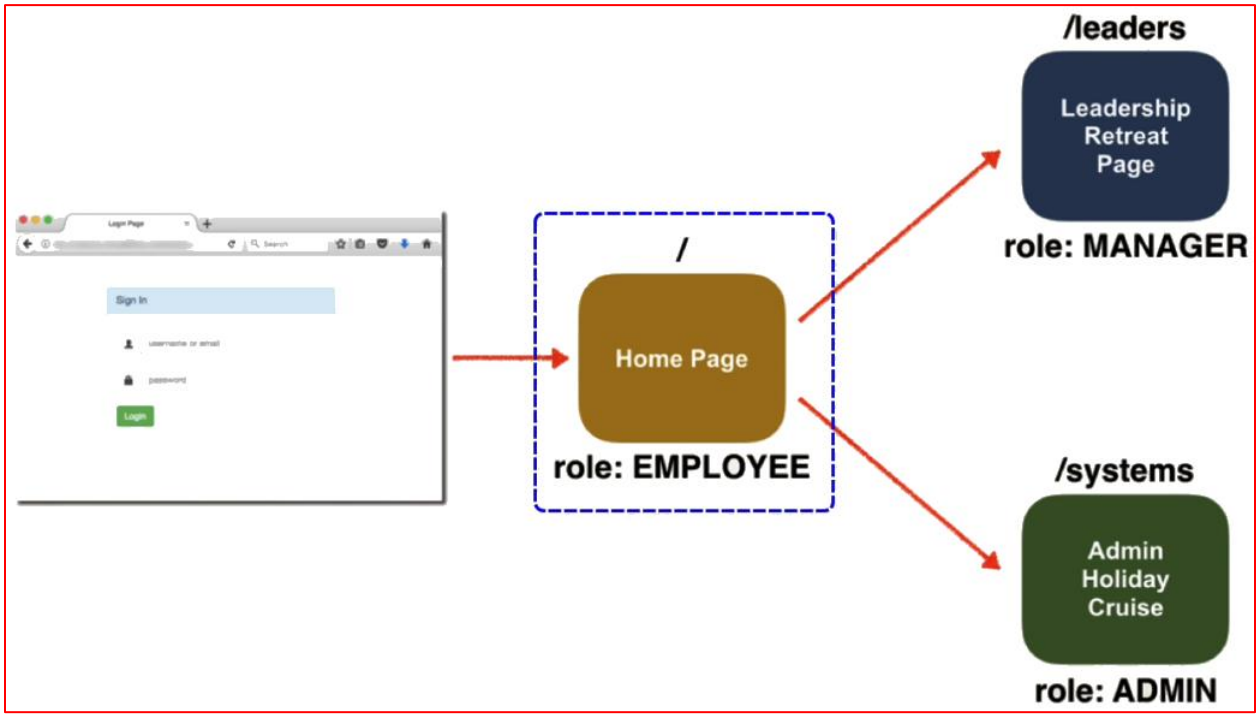
# Spring MVC Security - Restrict URLs Based on Roles – Coding

## Creating Access Controls and Building Web Pages

### *Introduction*

In this lesson**,** we are going to dive into creating a web page with specific access controls for different user roles. We'll build on a homepage with links pointing to various pages, each with different access restrictions.




### *Building the Homepage*

Let's begin by opening the homepage file. We will include links that will direct users to `/leaders` for the leadership page and `/systems` for the admin holiday cruise. In the center of our homepage, we will create these href links.

Consider the following user roles:

- John, Employee
- Mary, Manager
- Susan, Admin

We'll write code to display links that will point to the given pages and restrict access based on these roles.

*Development Process Overview*

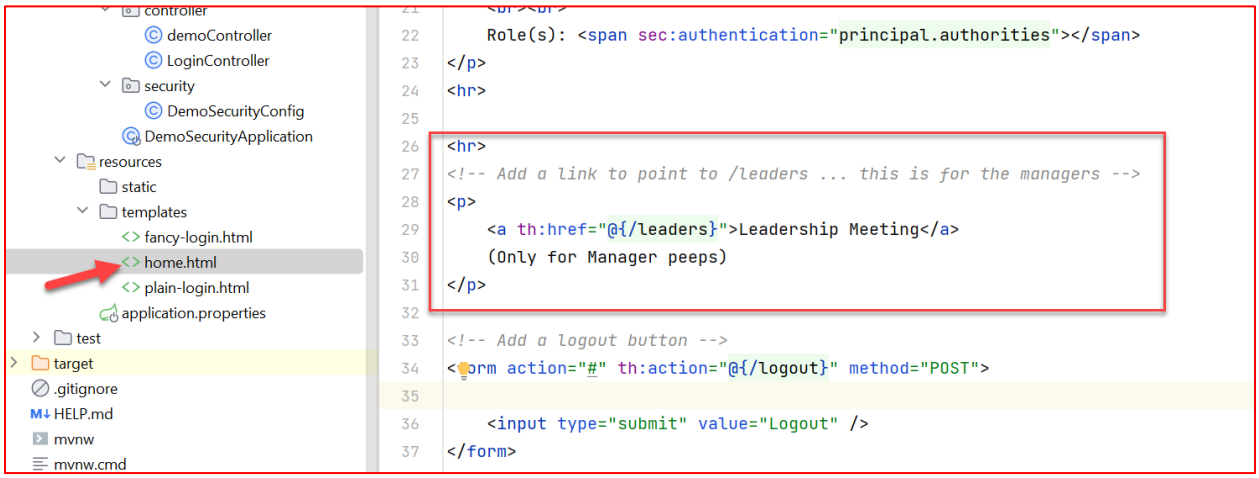
1. **Create Supporting Controller Code and View Pages:** We'll design the layout and basic functionality.
2. **Restrict Access Based on Roles:** We'll establish security measures to grant or deny access depending on user roles.

*Updating the Homepage*

Let's write a quick note to keep ourselves on track. We will add a link to point to `/leaders` for managers. Our desired output will be an href link on the actual page pointing to the leadership meeting.

Here's an example of how we'll set up the link:

```
<p>
  <a href="...">Leadership meeting</a> (only for manager peeps)
</p>
```




Please note, the text inside the parentheses is cosmetic only and doesn't impose any security restrictions. It's for educational and display purposes.

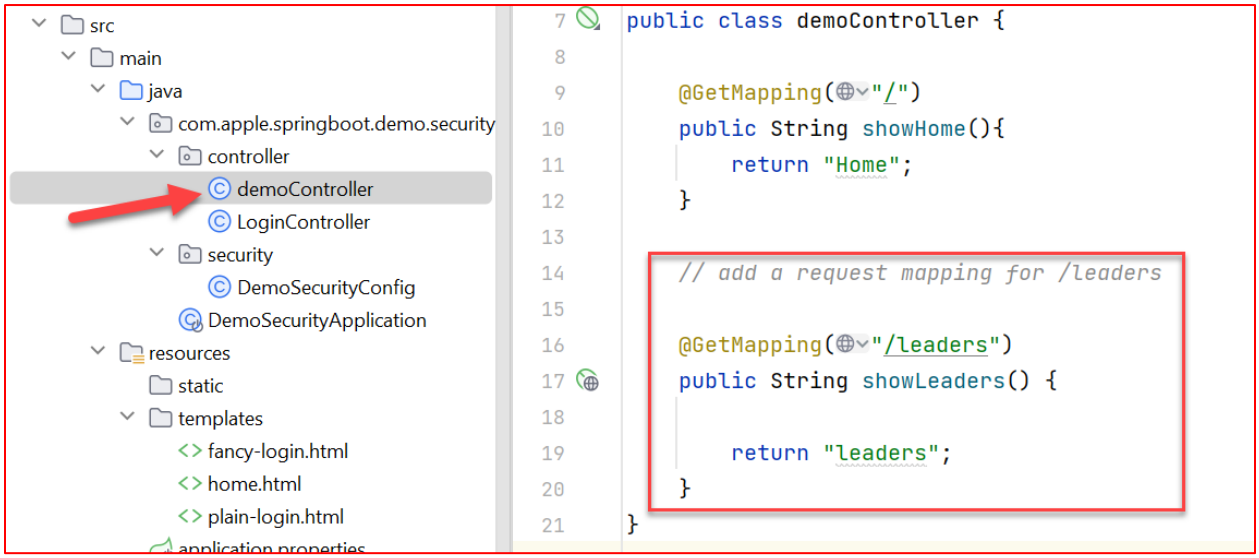
Now, we'll write the code in the href to actually point to `/leaders`. With this update, our homepage is ready.

*Creating Controller Code*

Moving on, we'll create the controller code and view code for `/leaders`. Let's dig into our source code, and open up `DemoController.java`. I will add another request mapping for `/leaders`.

Here's a simplified representation:

```
@GetMapping("/leaders")
public String showLeaders() {
  return "leaders";
}
```

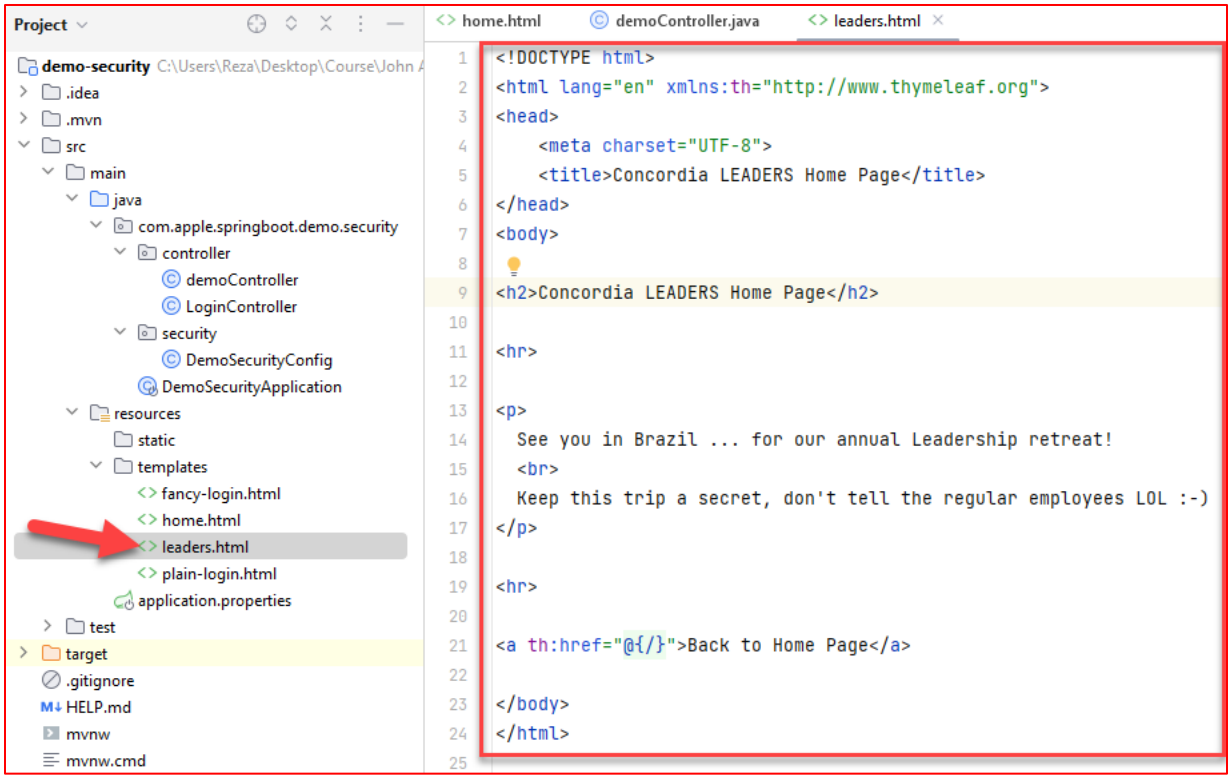



Then we need to create `src/main/resources/templates/`**`leaders.html`**

We'll set up a title for this page and provide some secret information accessible only to leaders. For example:

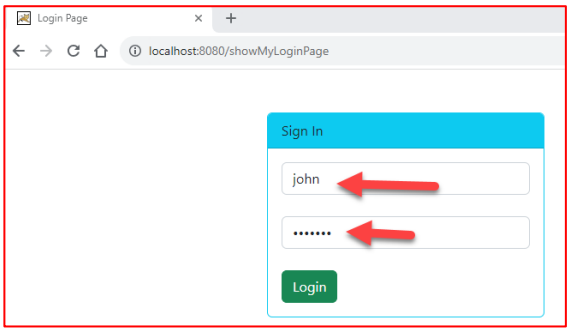
```
<h2>Concordia LEADERS Home Page</h2>
<p>See you in Brazil for our annual Leadership retreat. Keep this trip a secret.
Don't tell the regular employees about it, only for leaders.</p>
```

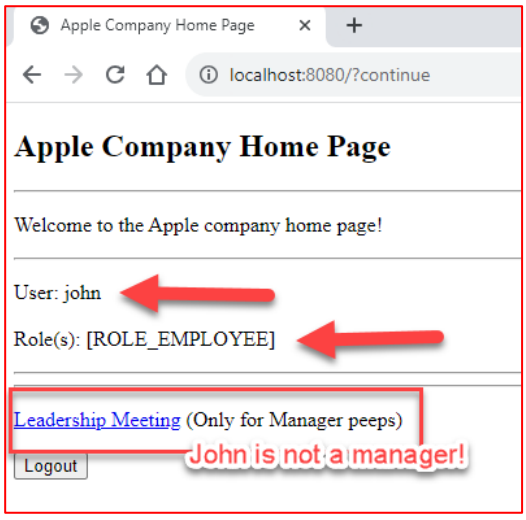
And finally, we'll add a link back to the homepage.




*Testing Access Controls*

We have a security problem. John is not a manager, but he can still access the leadership page! Oh, we haven't restricted access based on roles yet. Whew, more work to do!



*Restricting Access Based on Roles*

Let's define our access controls:

- `/` (root): accessible by anyone with the role of an employee.
- `/leaders/**`: accessible only by managers.
- `/systems/**`: accessible only by admins.

With this configuration in place, users will be restricted from accessing URLs for which they don't have the correct role.

Let's begin by opening the `DemoSecurityConfig.java` file.

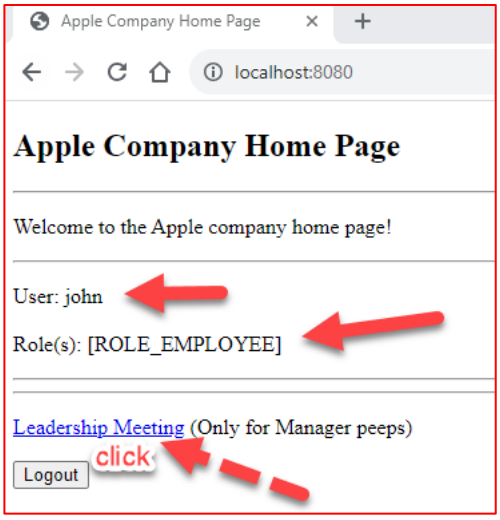
Here we will use the concept of `Matchers`, where we specify the actual path along with the role required to access it. For our first rule, we'll define the root path ("/") and associate it with the role of "`employee`." This configuration aligns with the example we initially discussed, allowing anyone with an "`employee`" role to access the homepage.

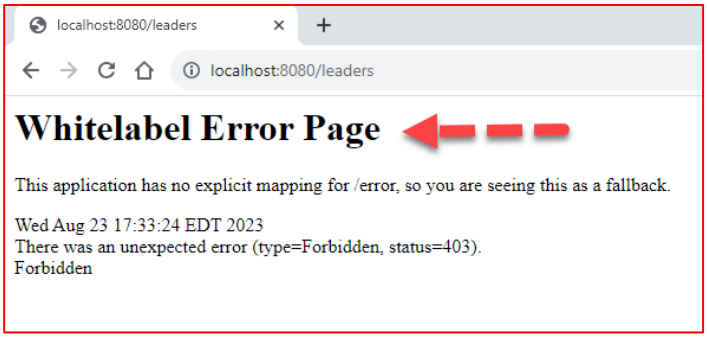
Next, we will copy and paste this line to accommodate other access paths. For the path "`/leaders/`," we will set the role as "`manager`." For the systems path, we'll define it as "`/systems/,`" where the "`**`" means all subdirectories, recursively. In this case, the role is "`admin`."



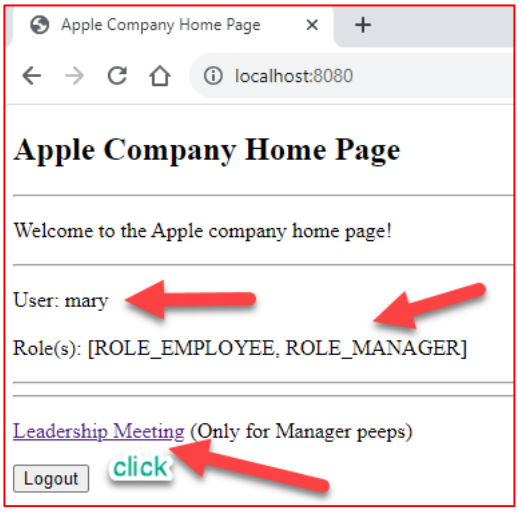

Now that we have these three matches in place, we'll save the code. If our server is already running, it will automatically reload the context and activate the new configurations, otherwise we re-run the project.

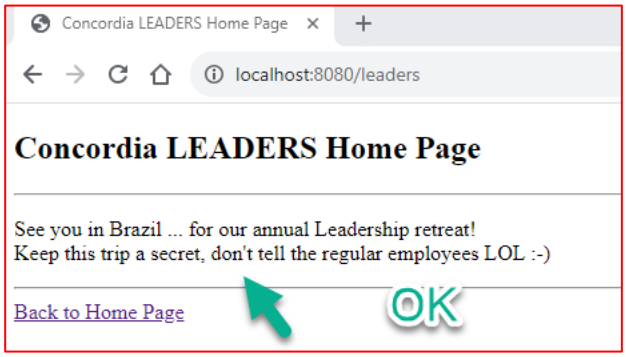
Let's test this setup by accessing the homepage as "`John`," who has the "`employee`" role. Attempting to access the leadership meeting for "`/leaders`" results in an authorization failure, just as expected. John does not have the correct access role to reach "`/leaders,`" and this is precisely what we intended.



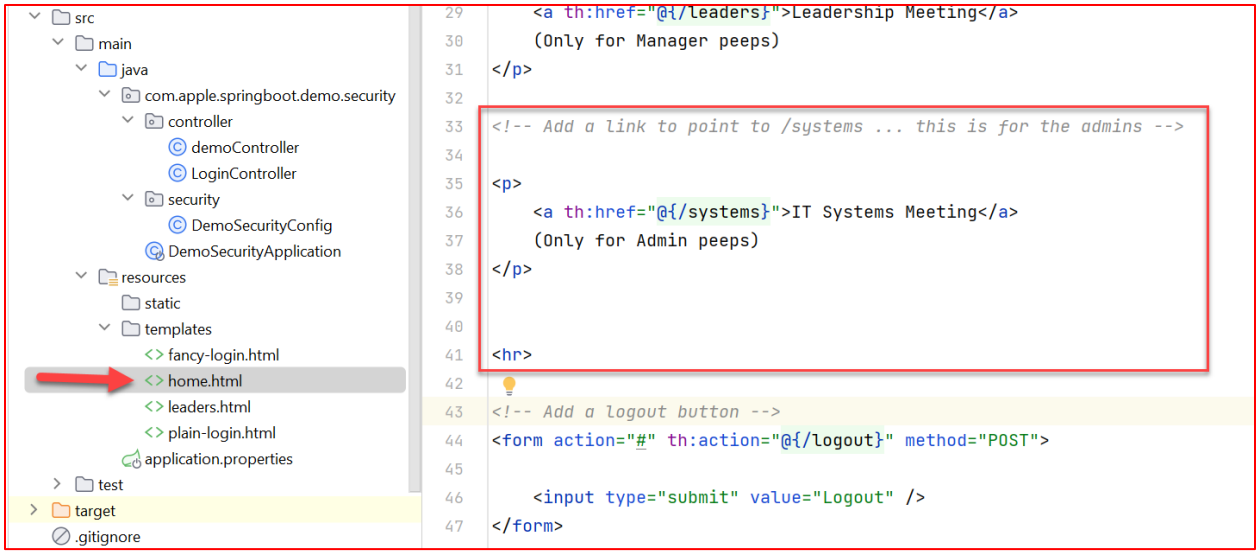




Next, let's log out and log in as "`Mary`," who has a "`manager`" role. We should find that she can access the `"/leaders"` link successfully.

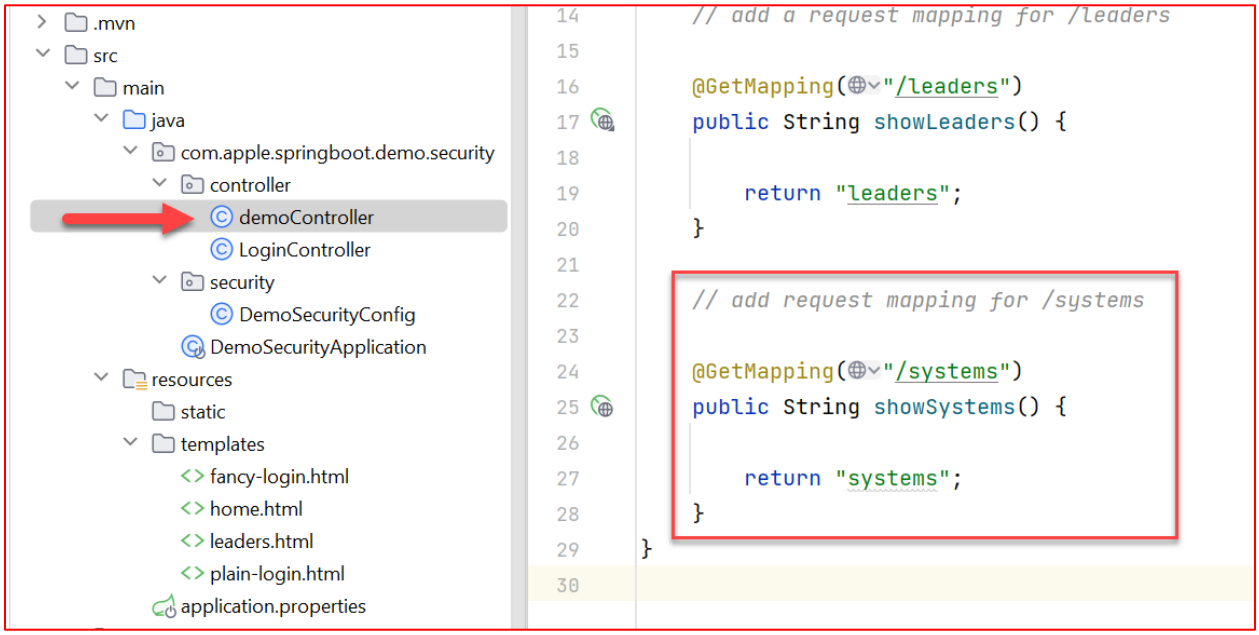


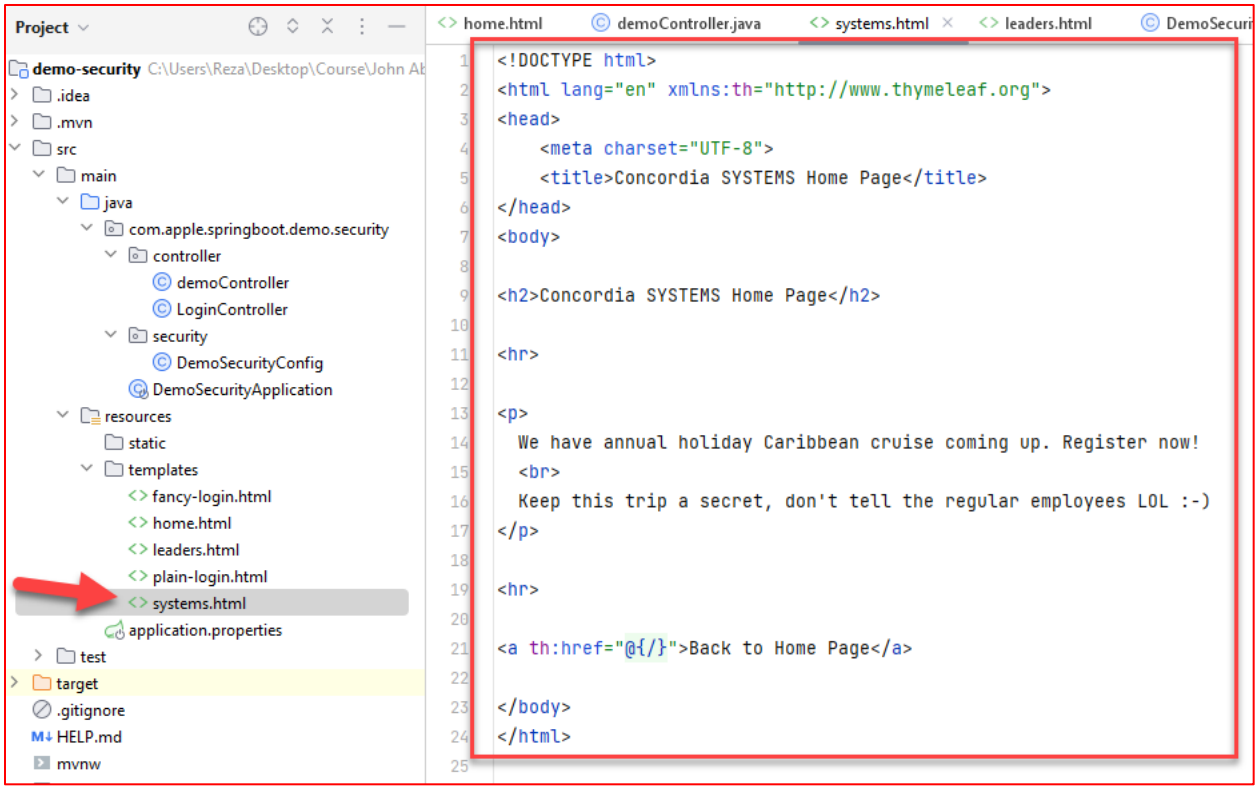

Now, let's continue our progress by focusing on the actual systems portion, specifically the IT systems meeting, accessible only to those with an "`admin`" role. I'll update the homepage by adding a link to `"/systems"` and writing a quick comment to remind myself that it's only for admins.



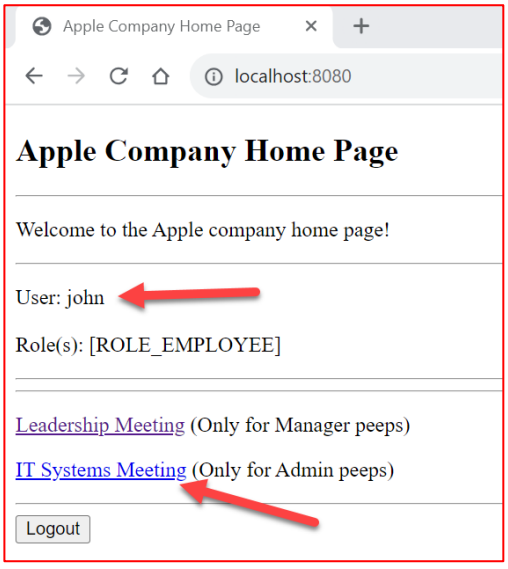

I'll proceed to update the `DemoController.java` with the request mapping for `"/systems,"` following the structure we used for `"/leaders."` We'll create a method called `showSystems` and return a view named "`systems`."

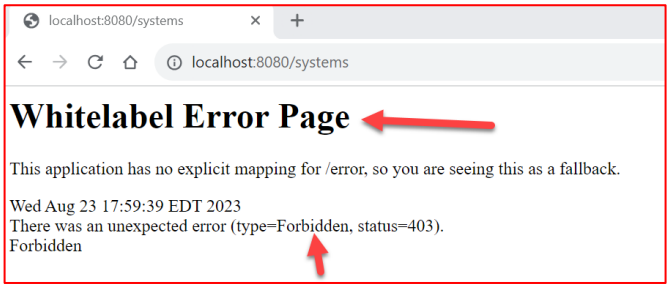


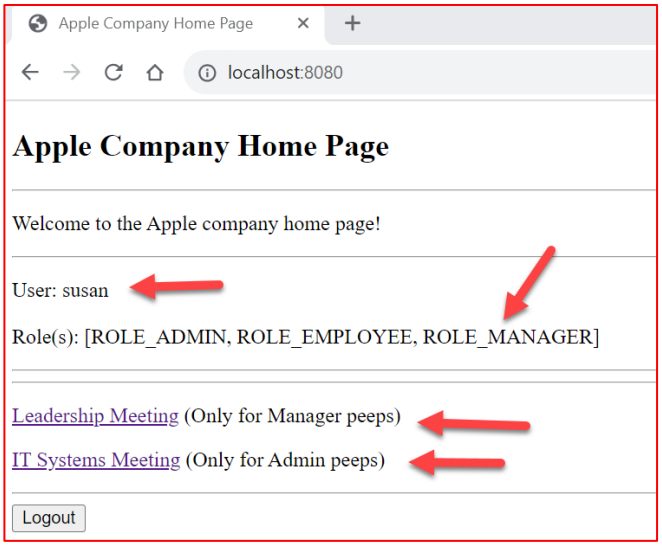

Next, we'll update the HTML content to match the theme of our systems homepage, including details about a secret annual holiday cruise in the Caribbean (keep this a secret!).

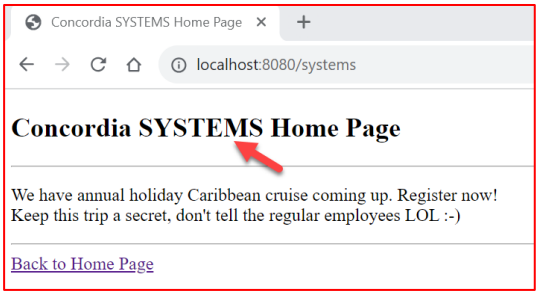



Now, logging in as "`john`," we find that he cannot access the systems page, as expected. Logging in as "`susan`," who has an admin role, we confirm that she can successfully access the IT systems meeting and learns about the secret Caribbean cruise.






## Summary

We've accomplished our goal of restricting access based on roles. We've defined different roles, set up paths, and put access restrictions in place, all through coding. This is an essential aspect of secure application development.